ASSIGNMENT -1.5

NAME : JAWWAD KHAN

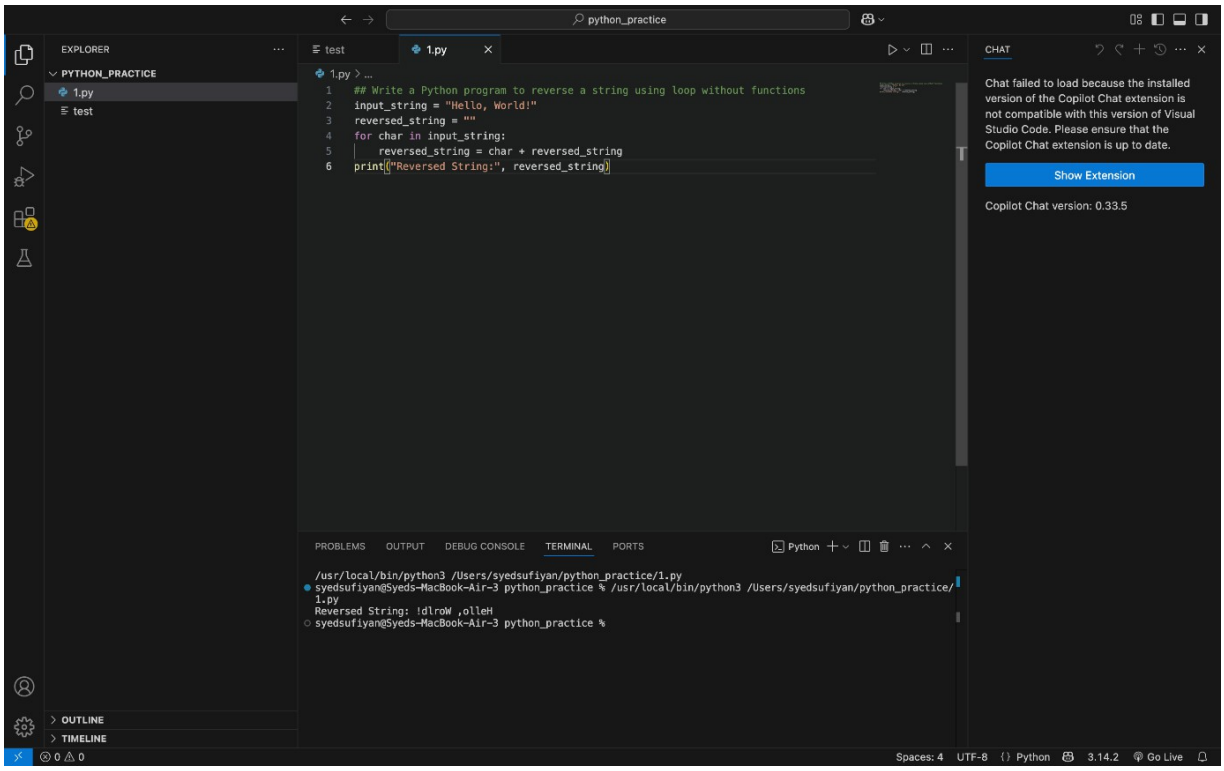
2303A51059

BATCH:29

TASK 1:

PROMPT: AI-GENERATED LOGIC WITHOUT MODULARIZATION (STRING REVERSAL WITHOUT FUNCTIONS)

CODE:



OBSERVATION:

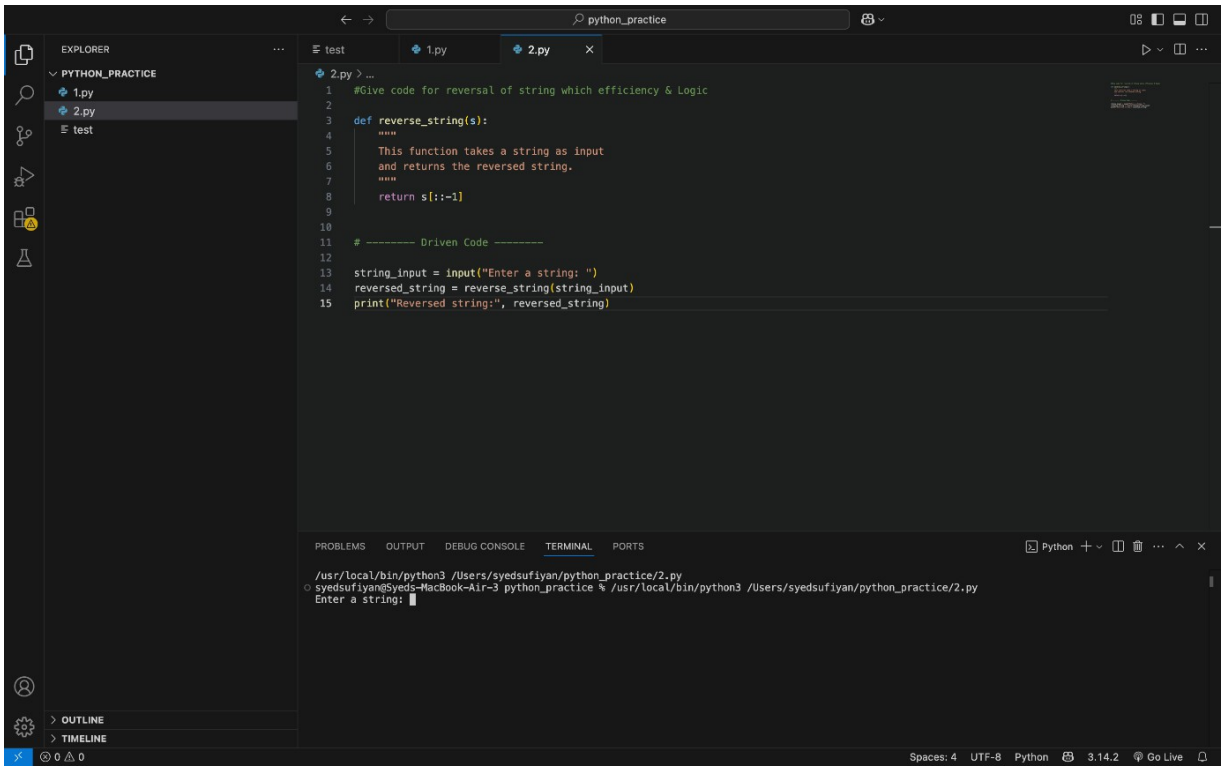
The program successfully reverses the given string using a simple loop without using any functions. Each character is added to the beginning of a new string, which gradually forms the reversed output. The output confirms that the logic works correctly by displaying the reversed string. This approach is easy to understand

and suitable for beginners learning basic string operations. However, for larger programs, a more optimized or modular approach would be better.

TASK:2

PROMPT: GIVE CODE FOR REVERSAL OF STRING WHICH EFFICIENCY & LOGIC OPTIMIZATION

CODE:



OBSERVATION:

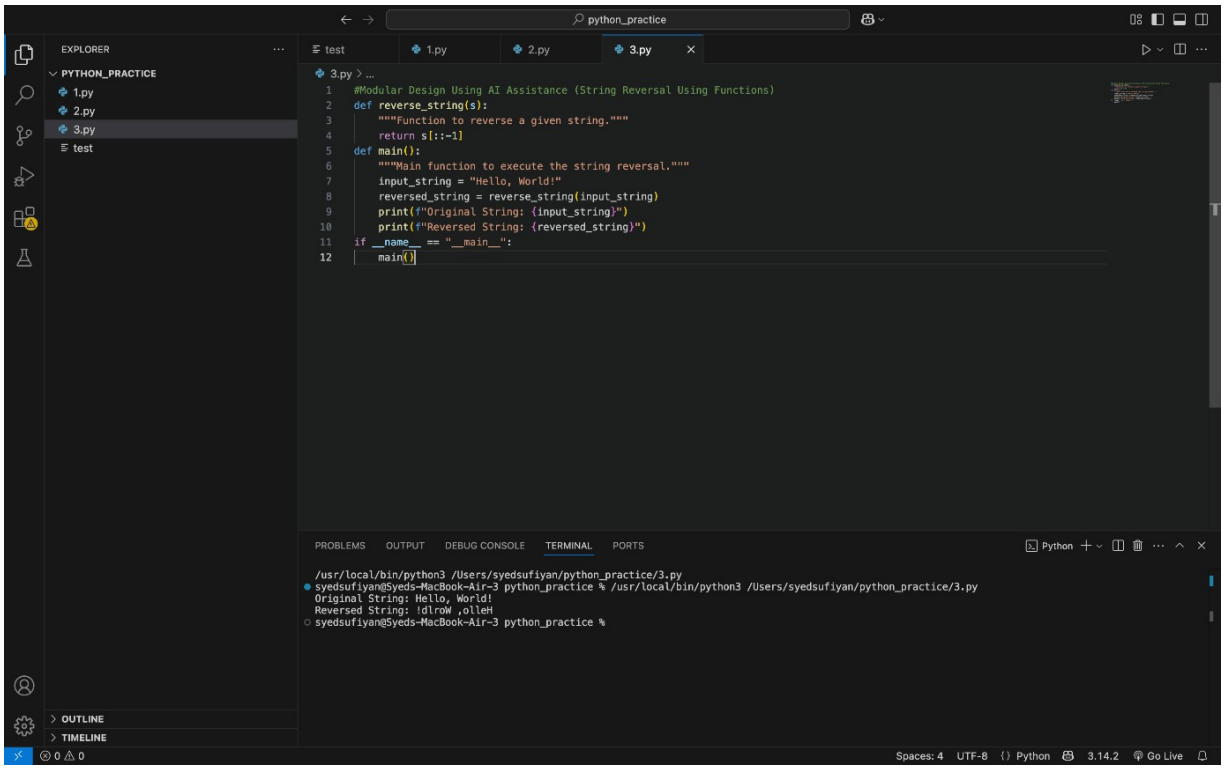
The function uses Python slicing to reverse the string in a single step. No extra variables or loops are used, which makes the code easy to read. The logic is efficient and executes faster than manual reversal methods. Overall, the code is clean, readable, and suitable for review by other developers. The optimized approach reduces unnecessary

operations and improves performance. It follows Python best practices, making the code more maintainable and reliable.

TASK:3

PROMPT: MODULAR DESIGN USING AI ASSISTANCE (STRING REVERSAL USING FUNCTIONS)

CODE:



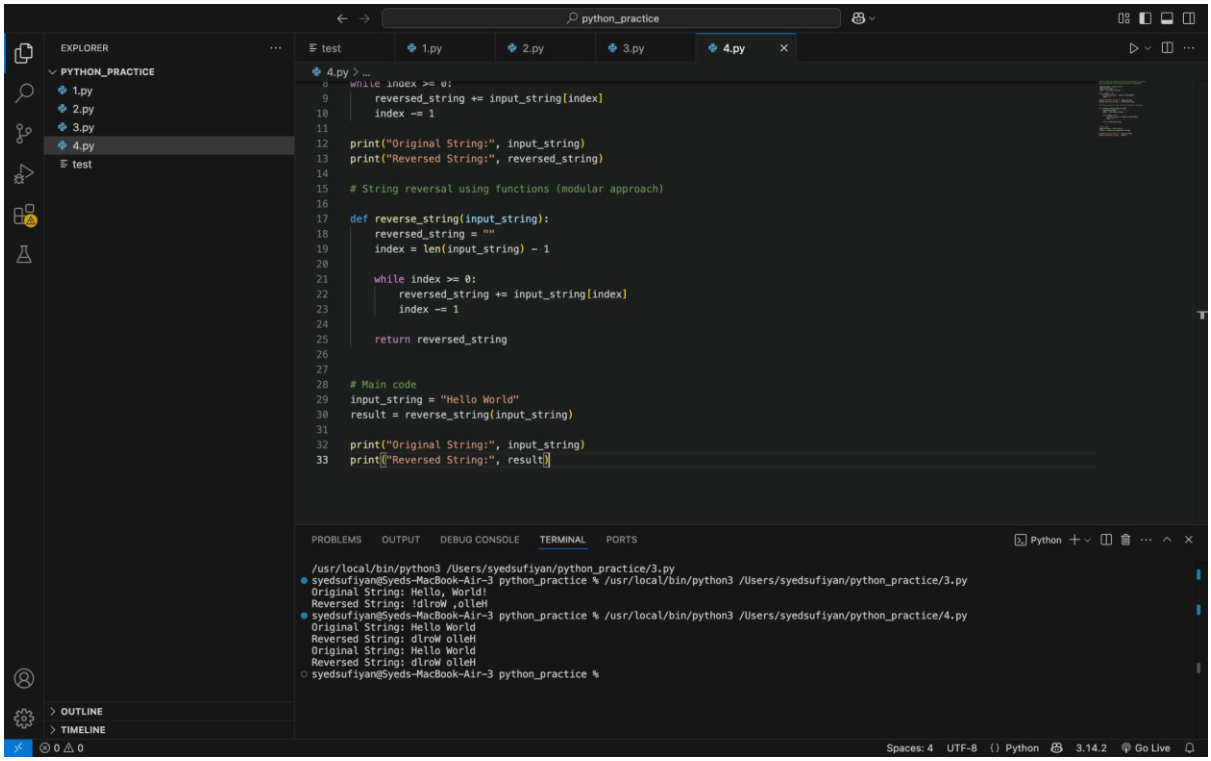
OBSERVATION:

This program follows a modular design by separating the string reversal logic into a reusable function. The use of clear function names and meaningful comments makes the code easy to understand and maintain. Since the reversal logic is written only once, it can be reused in multiple parts of the application without duplication. Overall, the structure improves readability, reusability, and makes future modifications simple.

TASK:4

PROMPT: COMPARATIVE ANALYSIS – PROCEDURAL VS MODULAR APPROACH (WITH VS WITHOUT FUNCTIONS)

CODE:



OBSERVATION:

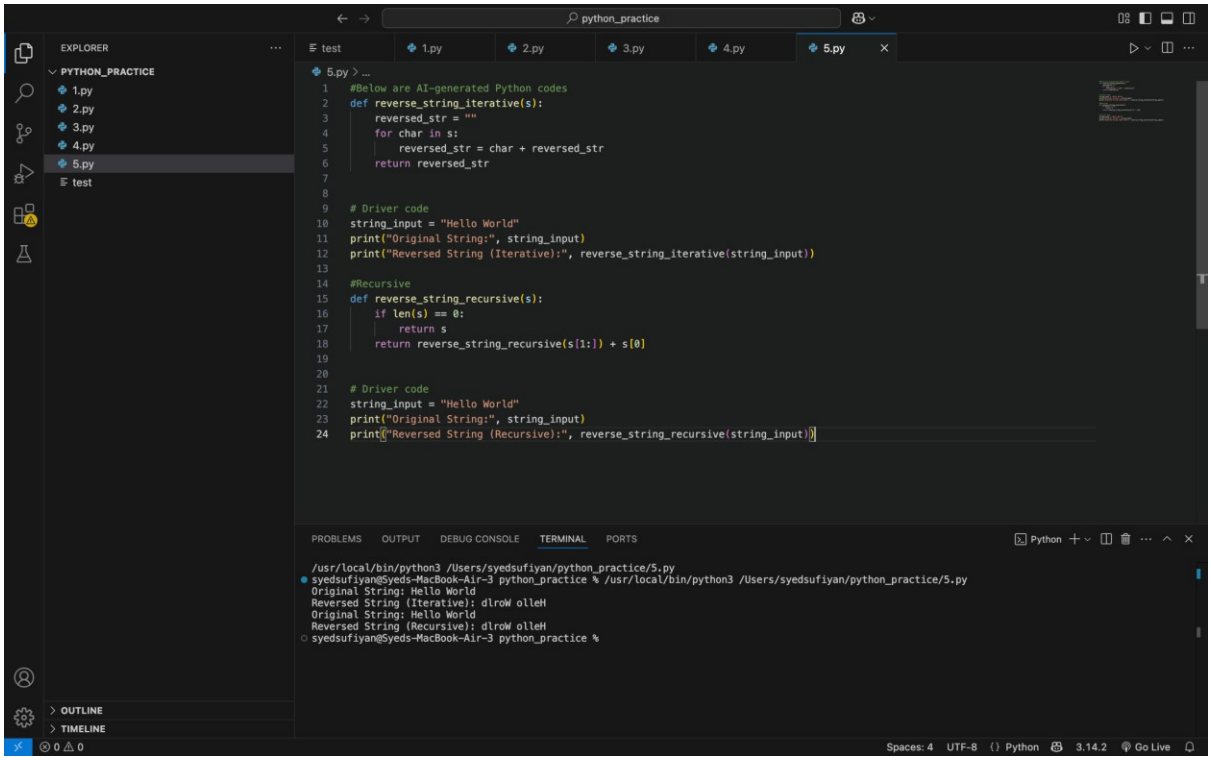
The procedural approach places all logic in one block, making the code harder to reuse and maintain. The modular approach separates logic into a function, improving clarity and structure. Functions allow easy reuse of code without duplication. Debugging is simpler in the modular approach because errors can be isolated. Overall, modular design is better suited for large and scalable applications

TASK:5

PROMPT:

AI-GENERATED PYTHON CODES ITERATIVE VS RECURSION

CODE:



OBSERVATION:

The iterative approach reverses the string by looping through each character, which makes the execution flow easy to follow but slightly slower due to repeated string concatenation. The recursive approach breaks the problem into smaller parts, which is conceptually clean but uses more memory because of function calls and stack usage. Both methods have linear time complexity, but recursion adds extra overhead. For large input strings, the iterative approach is generally safer and more efficient. The recursive method is better suited for

learning and understanding recursion rather than performance-critical applications.